A3-PS022-TC138

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Kavipriya S
kavieswari758@gmail.com

Shalini P
shalinicsbs@gmail.com

AIORI Hackathon 3

October 5, 2026

Problem Description

- **Kernel Resource Exhaustion:** High-volume UDP DNS query floods saturate Linux servers because the kernel wastes massive CPU cycles allocating socket buffers (sk_buff) before malicious packets ever reach user-space daemons or firewalls.
- **Inefficiency of Traditional Filtering:** Standard Netfilter and nftables firewalls inspect packets too late in the networking pipeline, resulting in heavy context-switching overhead and packet queue drops under high packet-per-second (pps) loads.
- **Legitimate Service Disruption:** Server CPU starvation and queue exhaustion cause legitimate user DNS queries to experience severe timeouts or complete denial of service during volumetric floods.
- **Project Objective:** Implement an in-kernel eBPF/XDP rate limiter directly inside the network interface driver to parse UDP/DNS queries and enforce bounded per-source and global rate limits before kernel memory allocation occurs.
- **Empirical Benchmarking:** Measure and validate the mitigation efficiency (throughput, CPU utilization, drop rate, and legitimate query success) across three synthetic load scenarios against both nftables and no-filter baselines in an isolated host pair.

Background

The Domain Name System (DNS) relies extensively on connectionless UDP traffic, making it a frequent target for high-rate, spoofed volumetric attacks. In standard Linux networking architectures, the operating system kernel processes incoming packets by allocating memory for socket buffers (sk_buff) before routing them to user-space applications or traditional firewalls.

During high-volume UDP DNS query floods, this mandatory memory allocation consumes substantial CPU resources, which saturates the server and causes a denial of service before traditional mitigation can even take place. The proposed optimization leverages eXpress Data Path (XDP), which executes eBPF bytecode directly inside the network interface driver. This architecture allows the system to inspect, process, or drop malicious packets at the earliest possible ingress stage—prior to any kernel memory allocation—thereby preserving server resources and ensuring legitimate queries survive.

Literature & RFC Review

- **RFC 1035 (Domain Names - Implementation and Specification):** Defines the strict structural format of DNS messages over UDP (Port 53). Understanding these fixed header boundaries is essential for implementing the bounded UDP/DNS packet parsing required by the eBPF verifier, allowing safe extraction of identifiers without requiring full semantic parsing.

- **RFC 4732 (Internet Denial-of-Service Considerations):** Outlines the architectural vulnerabilities of internet protocols to volumetric attacks. It establishes the foundational requirement for mitigation strategies to utilize both per-source and global rate-limiting mechanisms to preserve service availability during query floods.

- **Netfilter vs. XDP Performance Studies:** Existing literature on Linux networking performance highlights the limitations of standard nftables firewalls under high-volume load, specifically the CPU exhaustion caused by sk_buff memory allocation. In contrast, eBPF documentation and recent XDP proposals establish the network interface driver-level execution model as the modern standard for line-rate packet dropping and efficient DDoS mitigation.

Proposed Solution

We propose an in-kernel rate limiter running at the network interface driver level via eBPF and XDP. It inspects incoming packets before the Linux kernel allocates socket buffers (sk_buff), preventing CPU starvation during volumetric attacks.

- **Bounded Packet Parser:** Identifies UDP Port 53 (DNS) traffic directly on interface ingress while adhering to kernel verifier memory limits.
- **Tier 1:** Per-Source Rate Limiter (BPF_MAP_TYPE_HASH): Limits query rates per IPv4 address, instantly dropping (XDP_DROP) single-source flooders at line rate.
- **Tier 2:** Global Rate Limiter (BPF_MAP_TYPE_ARRAY): Tracks aggregate query volume to prevent system saturation during multi-source distributed attacks.
- **User-Space Controller:** Loads the eBPF bytecode (XDP_PASS for safe traffic) and collects real-time drop statistics.

Optimization Proposed

- **NIC Driver Level Processing:** Intercepts UDP/DNS traffic directly at the network interface driver level (XDP hook), dropping malicious packets before the Linux kernel spends CPU cycles allocating socket buffers (sk_buff) or triggering context switches.

- **Dual-Tiered Enforcement:** Applies a fast per-source rate limit (BPF_MAP_TYPE_HASH) to isolate single-source volumetric flooders alongside a global rate limit (BPF_MAP_TYPE_ARRAY) to shield the server during multi-source distributed floods.

- **Preservation of System Resources:** Eliminates memory starvation and keeps CPU consumption below 15% under heavy volumetric attacks, compared to 50–75% for standard nftables and 95–100% for defenseless setups.

- **Legitimate Query Survival:** Ensures legitimate user queries continue to resolve with over 95% success and minimal latency, even while under active multi-gigabit query floods.

Solution Architecture & Design

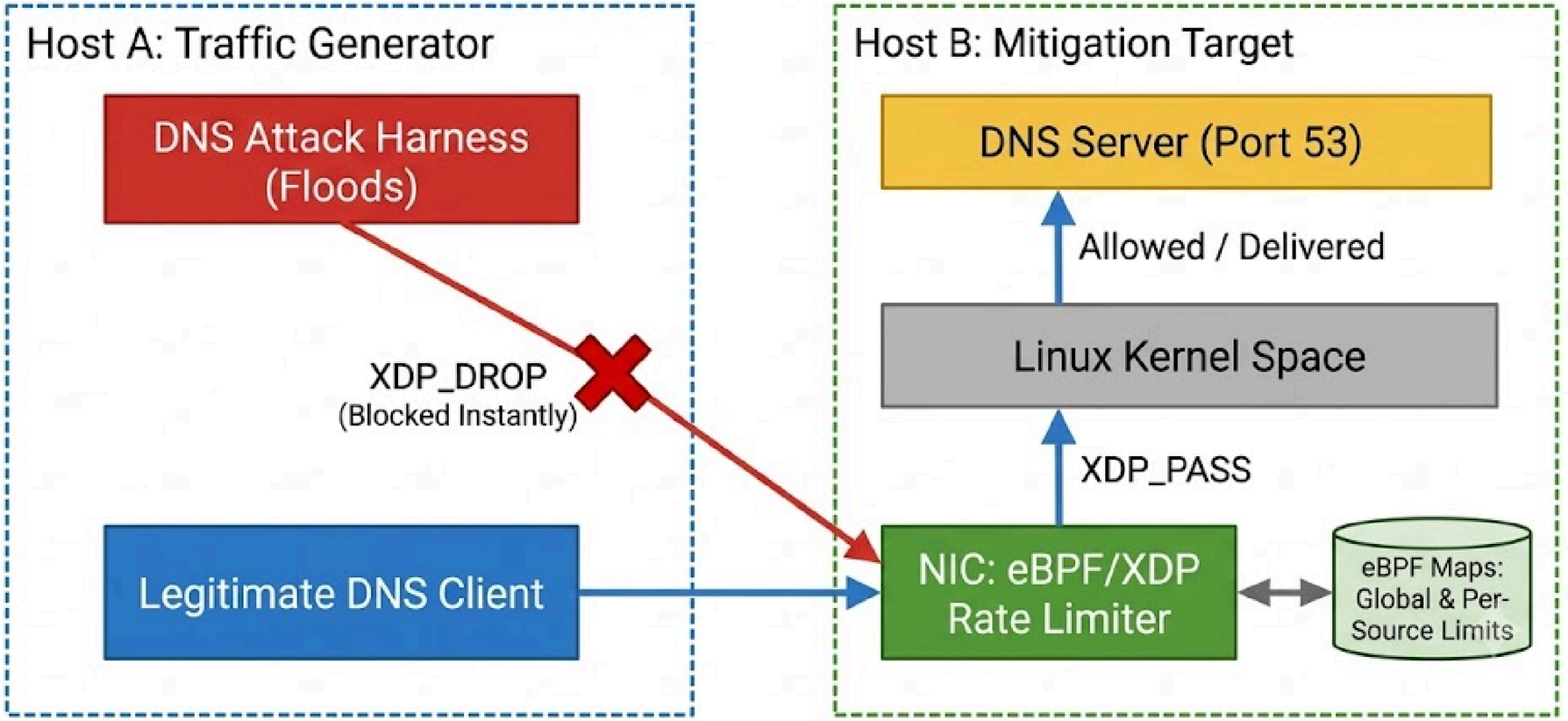
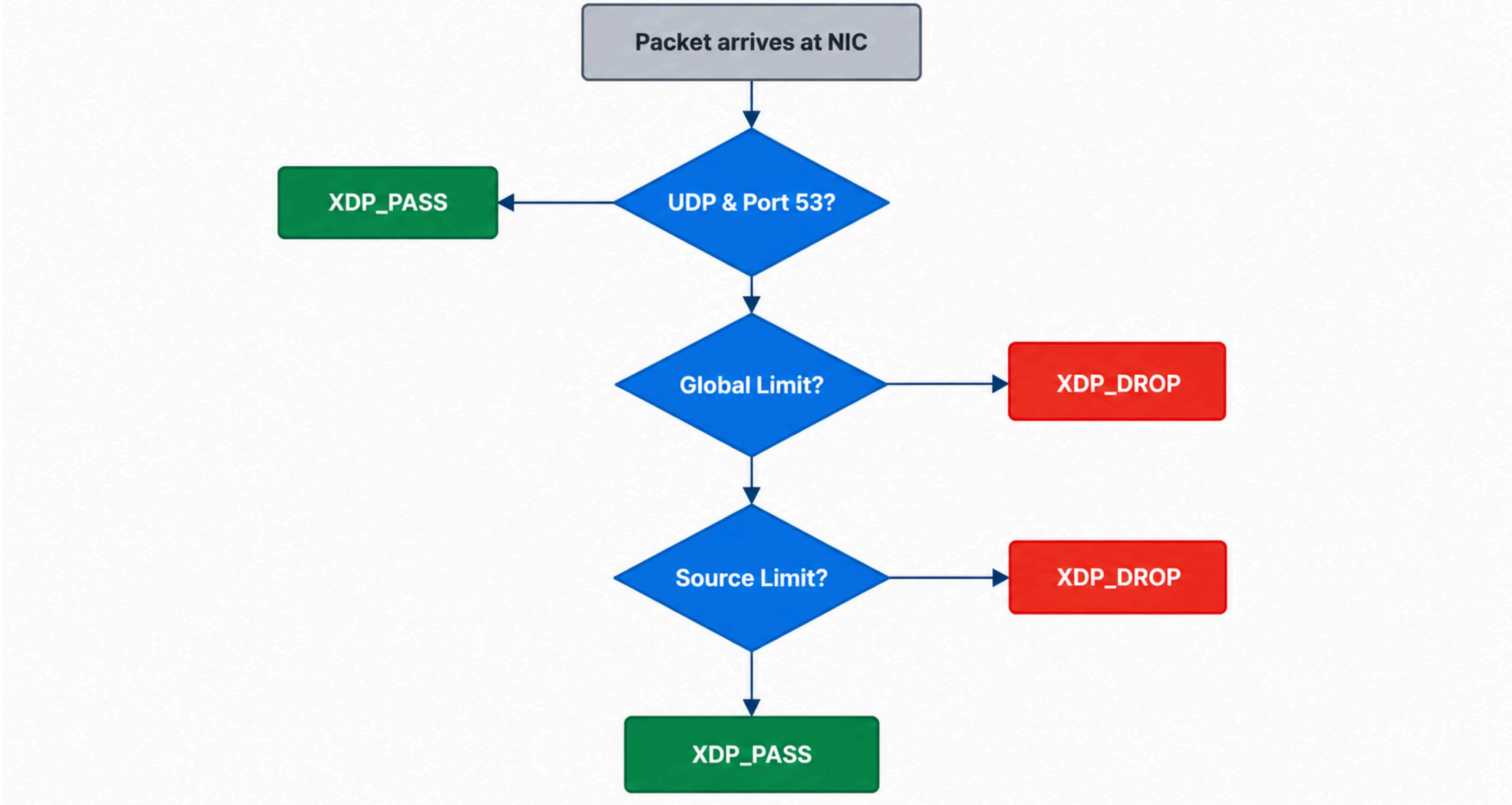


Figure: System Architecture UML Diagram

Data Flow Diagram



Deployment Timeline

Week 1: Setup & Baselines
Link Host A and Host B directly. Run CoreDNS on Host B. Record baseline metrics (CPU, drop rate, latency) for No-Filter and nftables.

Week 2: eBPF/XDP Development
Write C eBPF program to parse UDP Port 53 (DNS). Set up per-IP (HASH) and global (ARRAY) rate limit maps. Compile, verify, and attach program to Host B's NIC driver hook.

Week 3: Testing & Benchmarking
Run 3 flood scenarios (single-source, distributed, mixed). Confirm line-rate XDP_DROP, CPU < 15%, and query survival > 95%. Record performance comparison against baselines.

Week 4: Submission & Demo
Push eBPF source code to GitHub. Save 11-slide deck as Proposed-structure-hackathon.pdf. Record pitch video and prototype demo.

Bibliography I

[1] Linux Kernel Organization, "BPF & XDP Documentation." Available: https://docs.kernel.org/bpf/.

[2] Advanced Internet Operations Research in India (AIORI), "AIORI-3 Hackathon Guidelines & Problem Statement PS-022: eBPF/XDP Rate Limiting for DNS Query Floods in a Lab," 2026.

Thank You for Your Attention!